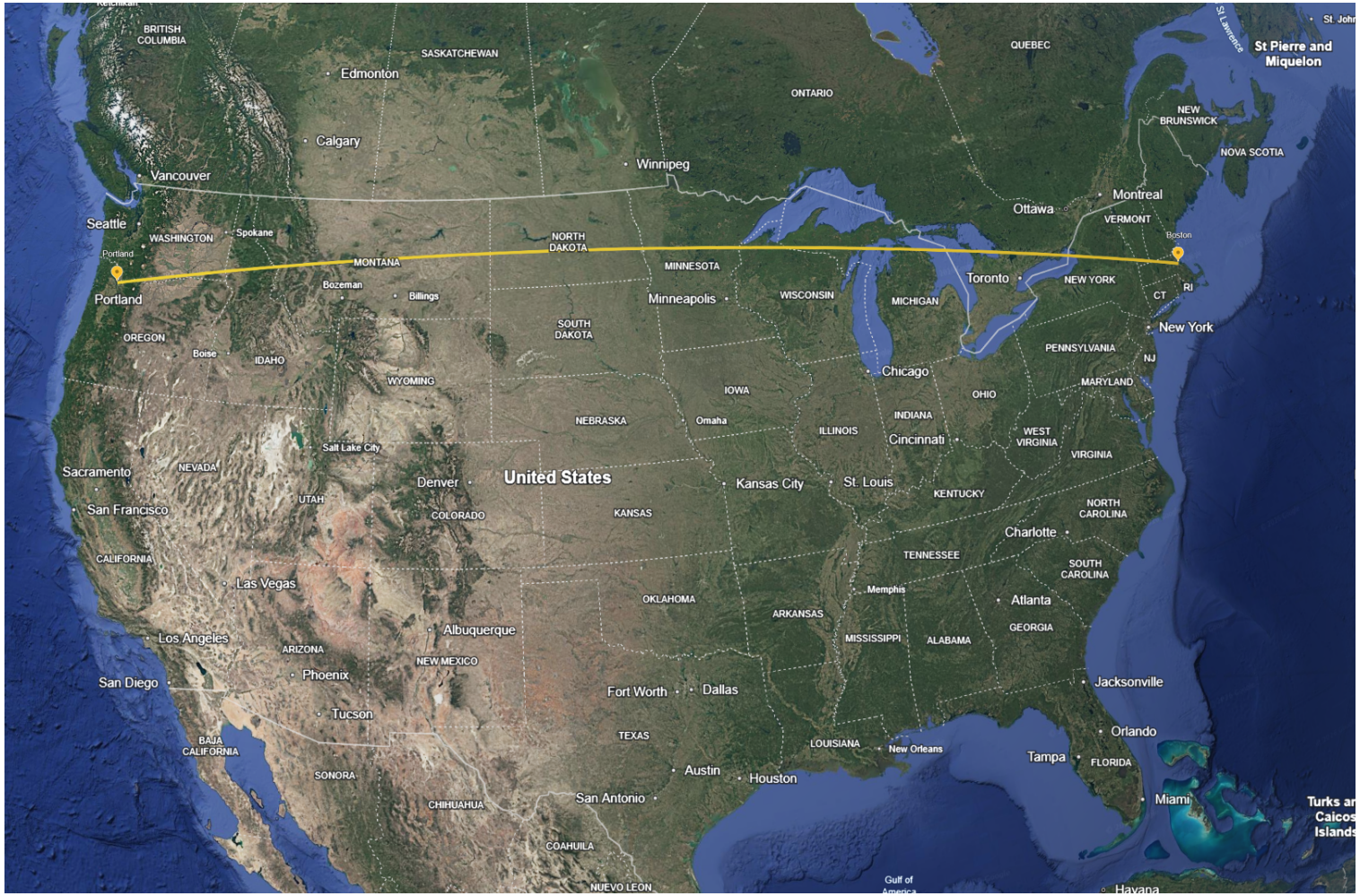
Logic for Programmers

Hillel Wayne



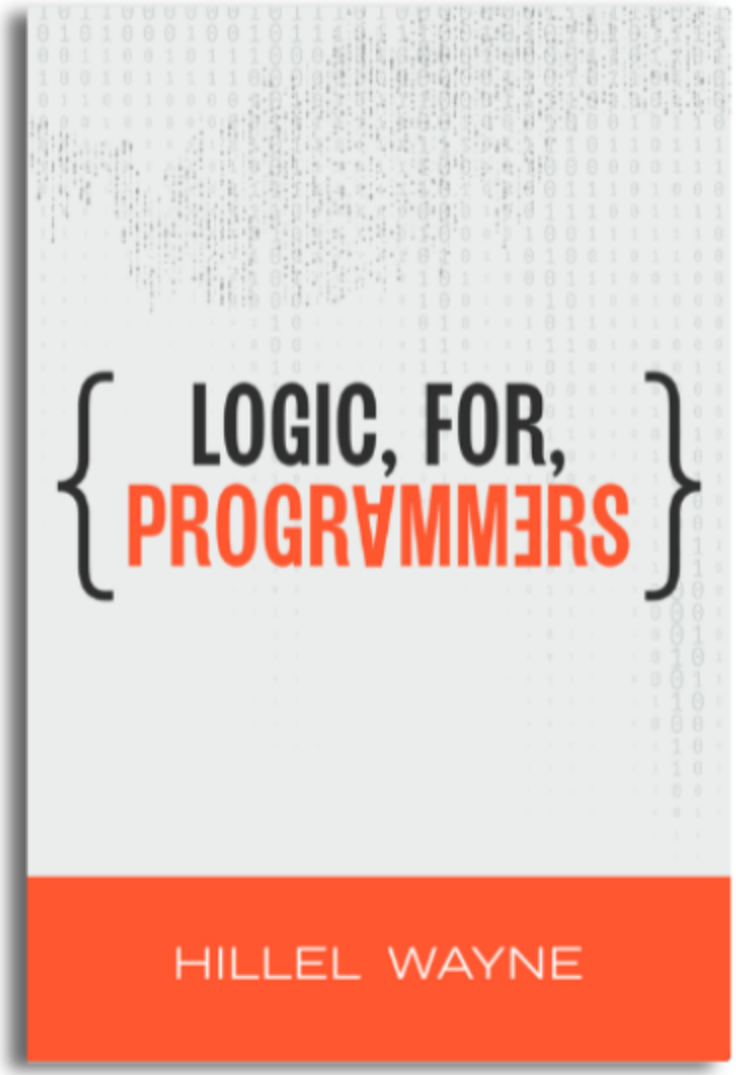

LARGE


BRITISH COLUMBIA
SASKATCHEWAN
Edmonton
ONTARIO
QUEBEC
St Pierre and Miquelon
NEW BRUNSWICK
NOVA SCOTIA
Calgary
Winnipeg
Vancouver
Ottawa
Montreal
VERMONT
Seattle
WASHINGTON
Spokane
Portland
NORTH DAKOTA
Boston
MONTANA
MINNESOTA
Toronto
NEW YORK
Bozeman
Billings
Portland
Minneapolis
WISCONSIN
MICHIGAN
CT
RI
SOUTH DAKOTA
New York
OREGON
Boise
IDAHO
PENNSYLVANIA
NJ
Chicago
WYOMING
IOWA
MARYLAND
OHIO
NEBRASKA
Omaha
INDIANA
ILLINOIS
Cincinnati
WEST VIRGINIA
Salt Lake City
VIRGINIA
Sacramento
NEVADA
Denver
United States
Kansas City
St. Louis
KENTUCKY
San Francisco
UTAH
COLORADO
KANSAS
NORTH CAROLINA
Charlotte
CALIFORNIA
TENNESSEE
SOUTH CAROLINA
Las Vegas
Memphis
OKLAHOMA
Atlanta
ARKANSAS
Albuquerque
GEORGIA
Los Angeles
MISSISSIPPI
ALABAMA
ARIZONA
NEW MEXICO
San Diego
Phoenix
Jacksonville
Fort Worth
Dallas
Tucson
Orlando
TEXAS
LOUISIANA
New Orleans
BAJA CALIFORNIA
Tampa
FLORIDA
SONORA
Austin
Houston
San Antonio
Miami
CHIHUAHUA
COAHUILA
NUEVO LEON
Gulf of America
Havana



Now for the possibility of a disjunctive proposition it is sufficient that either of its parts be possible. But for a disjunctive proposition to be impossible it is required that both parts be impossible.

!(A || B) == !A && !B



Logic for Programmers

A book about math, software, and using one to fix the other.

For the working programmer.

No math background required.

Some book topics

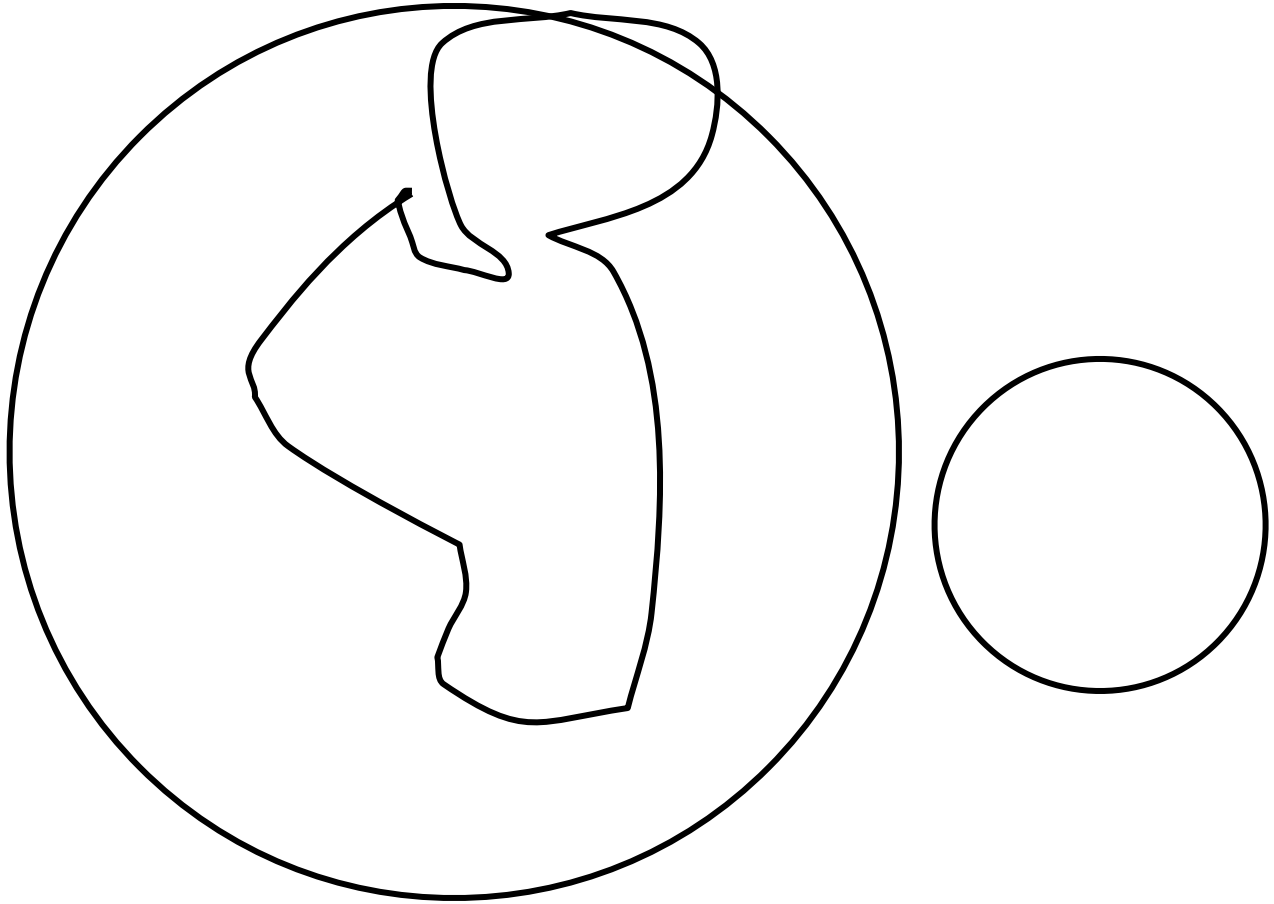
- Property testing
- Database theory
- Constraint solving
- Formal methods
- Prolog
- Replacement
- Ability-Guarantee Tradeoff
- Set optimizations
- SMT solving
- Decision Tables
- The nature of time

Some book topics

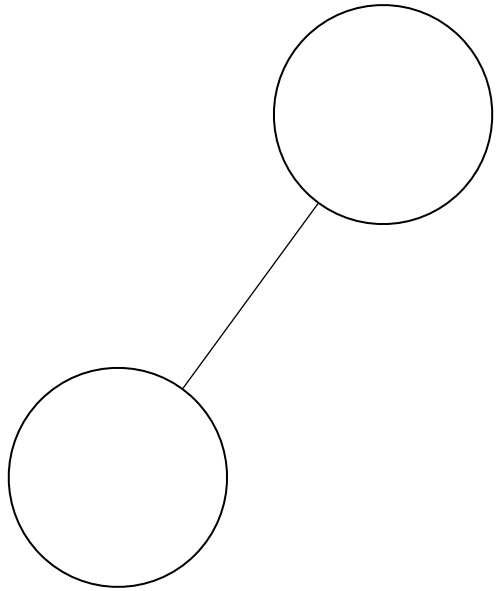
- Replacement

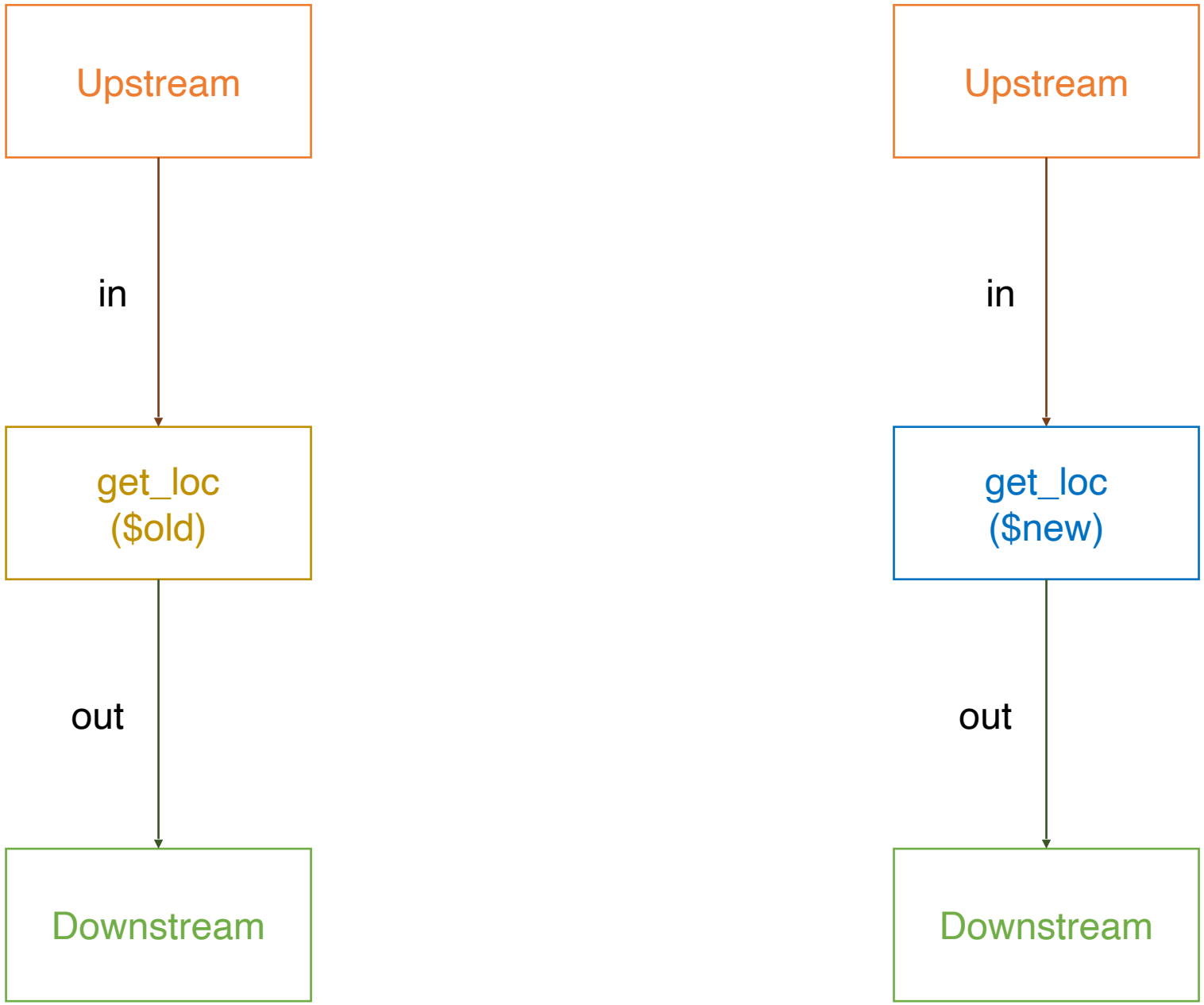
When can $new replace $old? (without breaking existing code depending on $old)

get_loc :: Device -> Location



We don’t know how people are using our API




Upstream
in
get_loc
($old)
out
Downstream
Upstream
in
get_loc
($new)
out
Downstream

Intuition for replacing APIs

$new can replace $old when workflows using $old don't break with $new

The Math

The TV is off, or on and not working.

The TV is off, or on and not working.

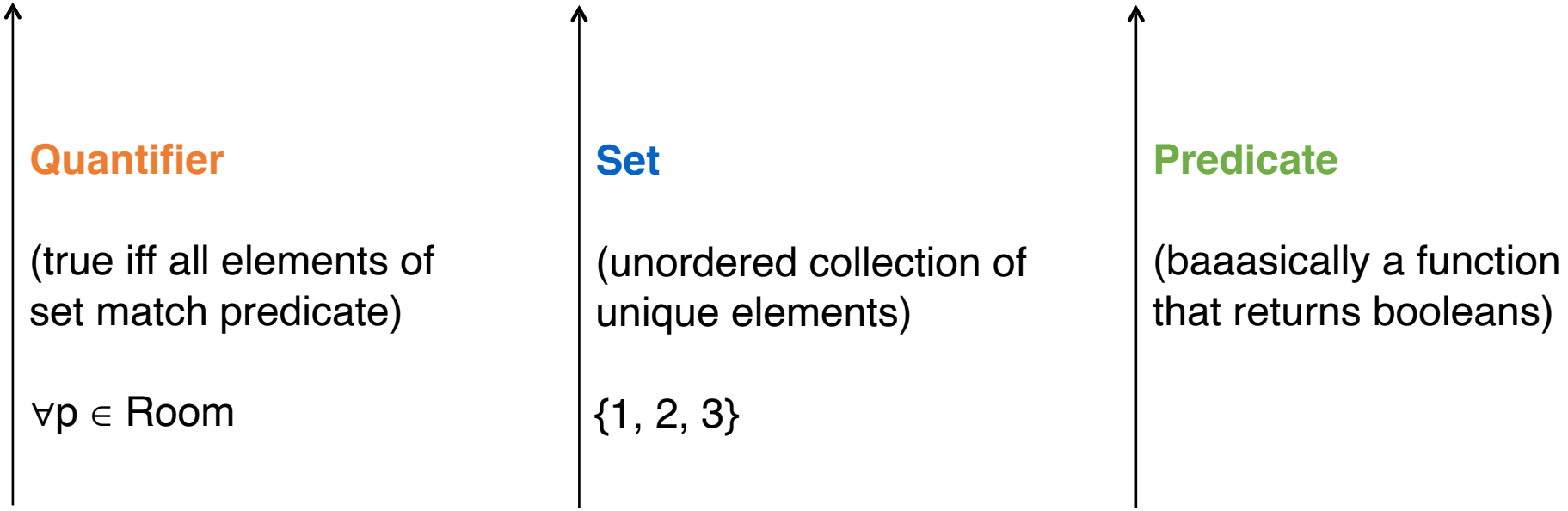
TVOff ll (TVOn && !TVWorking)

Everyone in this room is bald.

What we'll cover

- Sets
- Predicates
- Quantifiers
- Implication

Everyone in this room is bald.



all p in Room: Bald(p)

Expression

!(all p in Room: Bald(p))

some (∃) p in Room: !Bald(p)

all x: P(x) == !(some x: !P(x))

(Duality)

Everyone in this room named Hillel is bald.

Everyone in this room named Hillel is bald.

Set filter

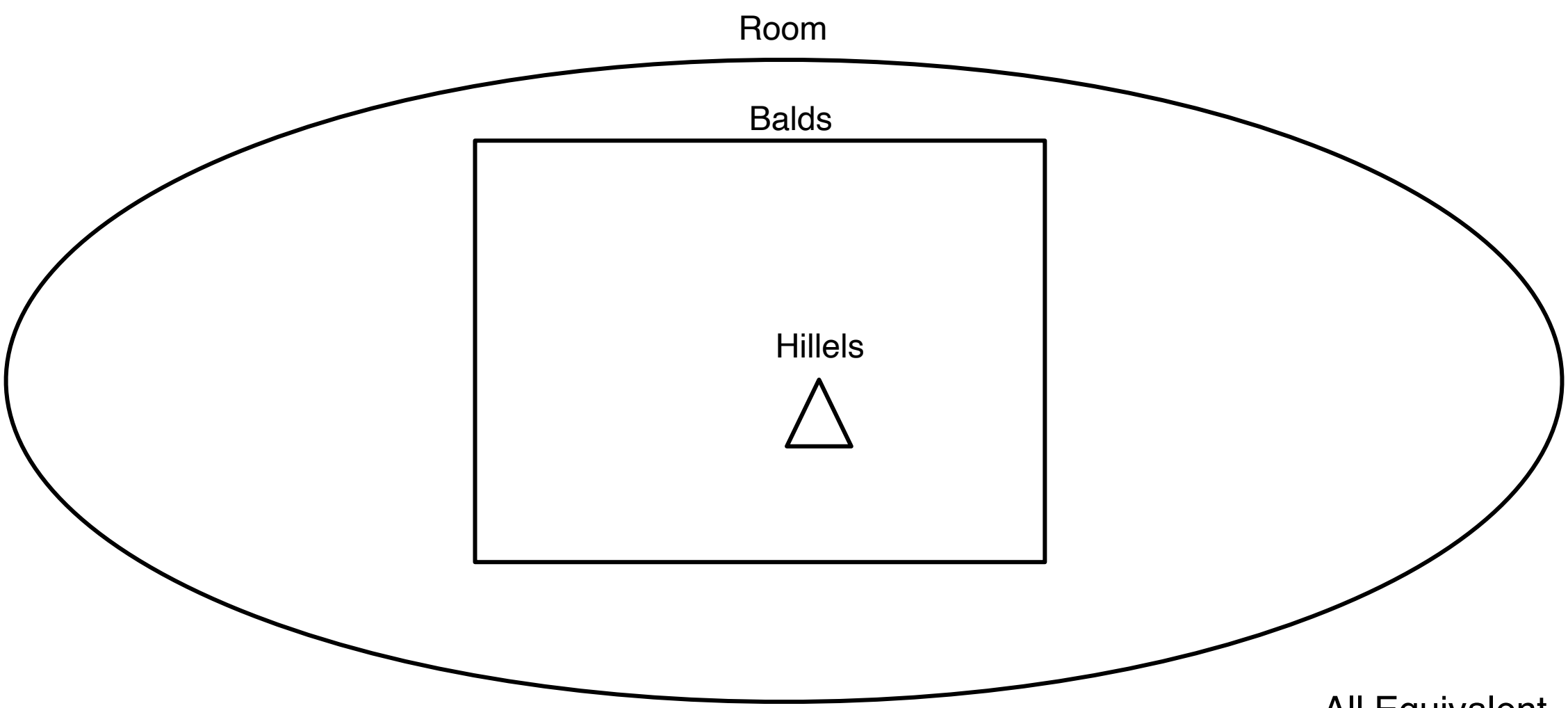
Hillels = {p in Room: Hillel(p)}

all p in Hillels: Bald(p)

all p in Room: !Hillel(p) ll Bald(p)

all p in Room: Hillel(p) => Bald(p)

Implication



{p in Room: Hillel(p)} subset {p in Room: Bald(p)}

all p in {p in Room: Hillel(p)}: Bald(p)

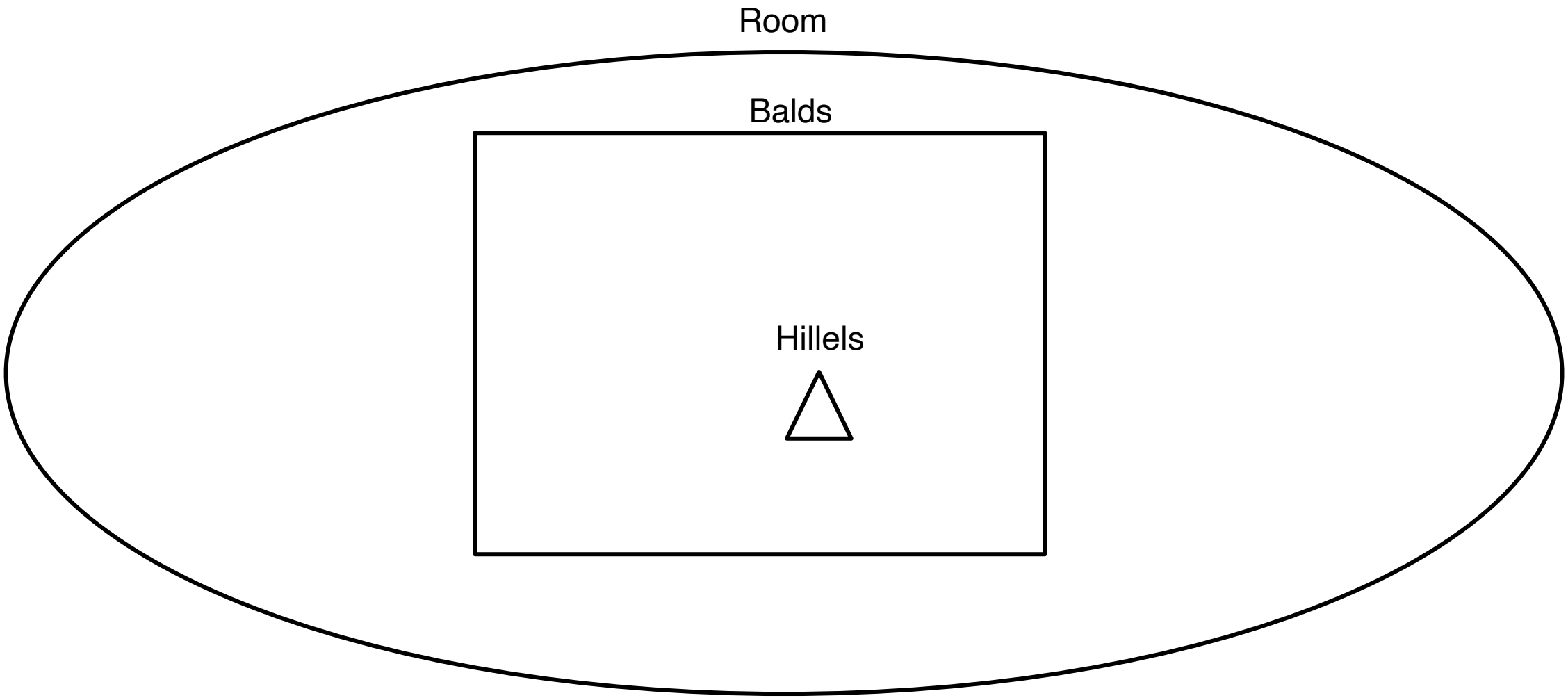
all p in Room: Hillel(p) => Bald(p)

All Equivalent

Subset

Quantifier

Implication



Hillels subset Balds && Balds subset Room

all p in Room:
  Hillel(p) => Bald(p) && Bald(p) => Room(p)

Hillels is subset of Room

all p: Hillel(p) => Room(p)

What if Hillel had a Mohawk?

!(all p in Room: Hillel(p) => Bald(p))

duality: all x: P(x) == !(some x: !P(x))

some p in Room: !(Hillel(p) => Bald(p))

implication: a => b == !a || b

some p in Room: !(!Hillel(p) || Bald(p)

De Morgan's law: !(a || b) == !a && !b

some p in Room: Hillel(p) && !Bald(p)

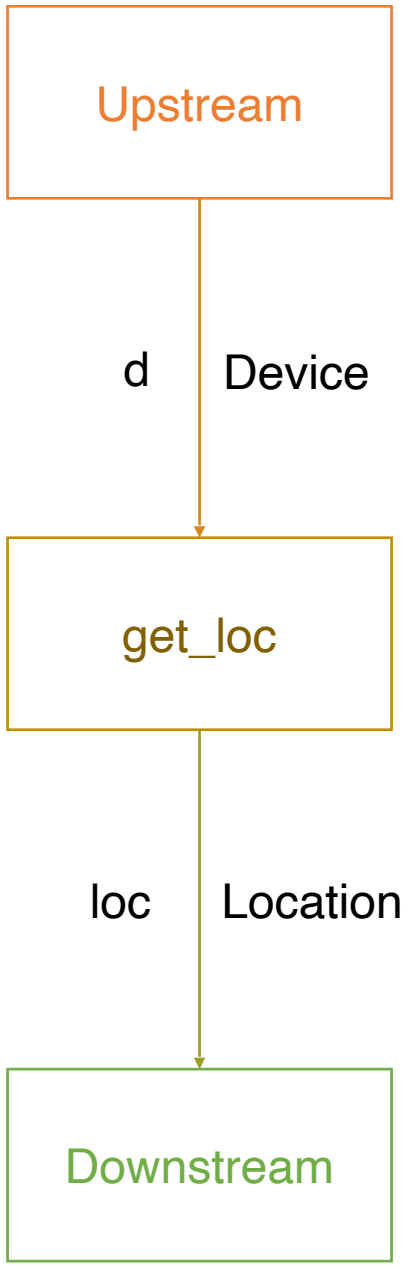
What we covered

- Sets: {1, 2, 3}
  unordered, unique collections
- Predicates: Foo(x)
  Boolean functions
- Quantifiers: all and some
  check predicates on sets

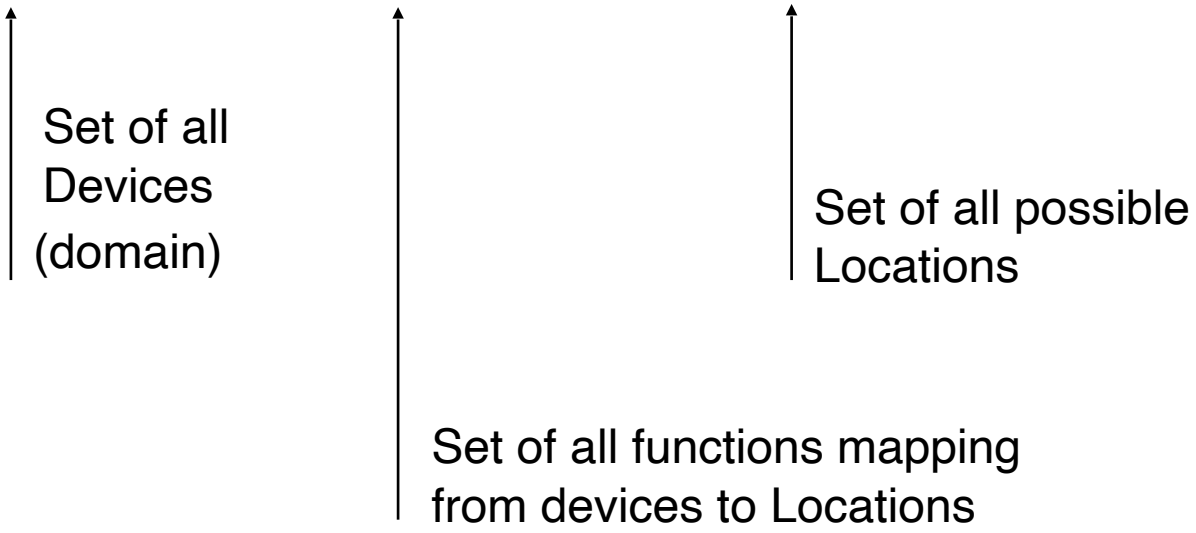
- Implication: a => b
  A weird operation that has a lot of properties we like

Intuition for replacing APIs

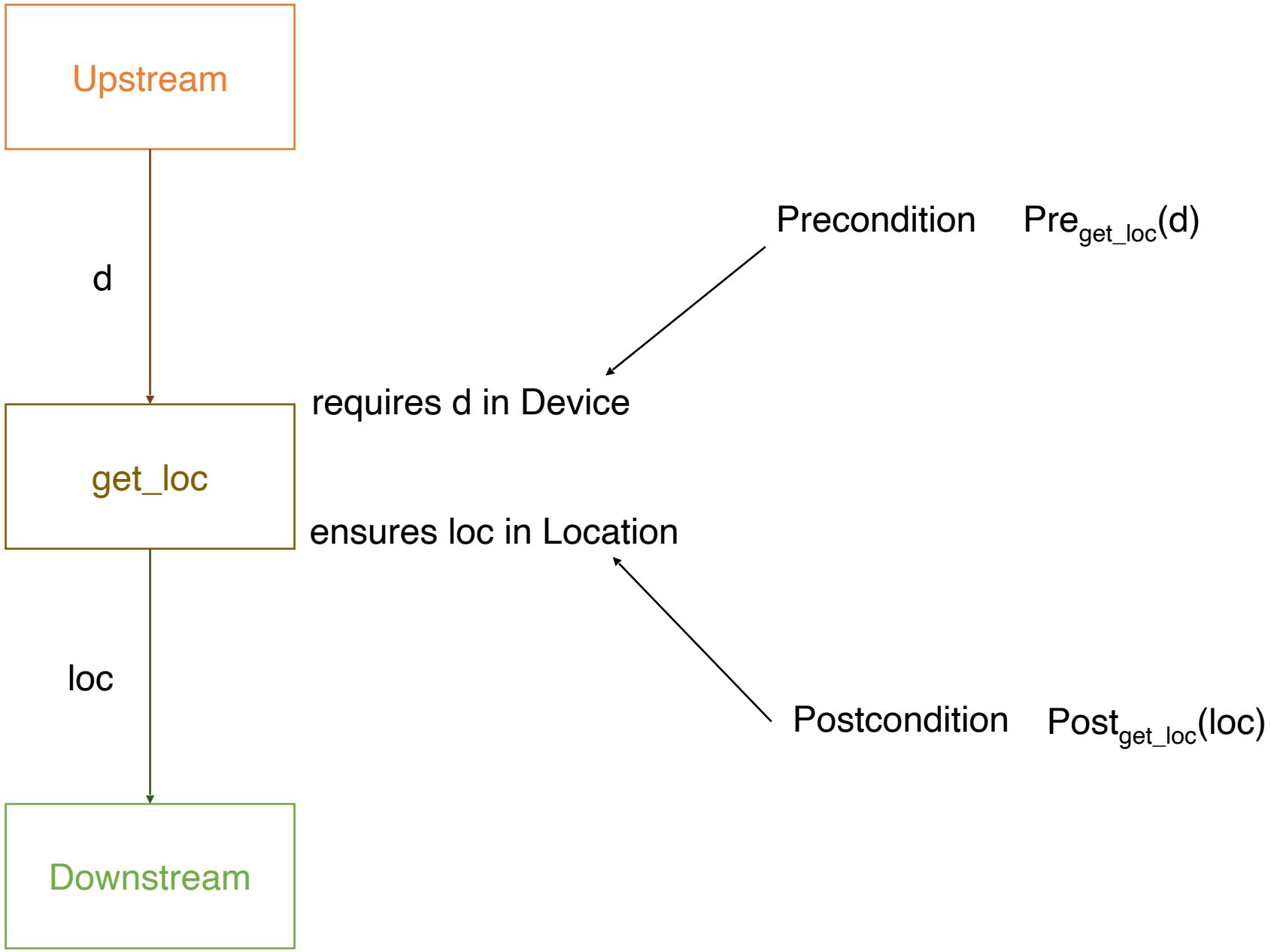
$new can replace $old when workflows using $old don't break with $new


Upstream
d
Device
get_loc
loc
Location
Downstream

get_loc :: Device -> Location



all d in Device: get_loc(d) in Location

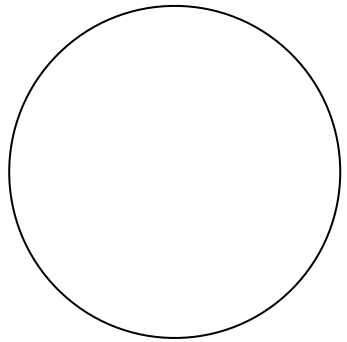

Upstream
d
get_loc
loc
Downstream
Precondition
$Pre_{get_loc}(d)$
requires d in Device
ensures loc in Location
Postcondition
$Post_{get_loc}(loc)$

get_loc :: Device -> Location

$Post_{get_loc}(loc)$ = loc in Location

$Post_{get_loc}(d, loc)$ =

1. loc in Location
2. LocatedIn(d, loc)
3. area(loc) ≤ 1 deciacre



Guarantee = all d in Device: $Post_{get_loc}(d, loc)$

get_loc()



get_loc()









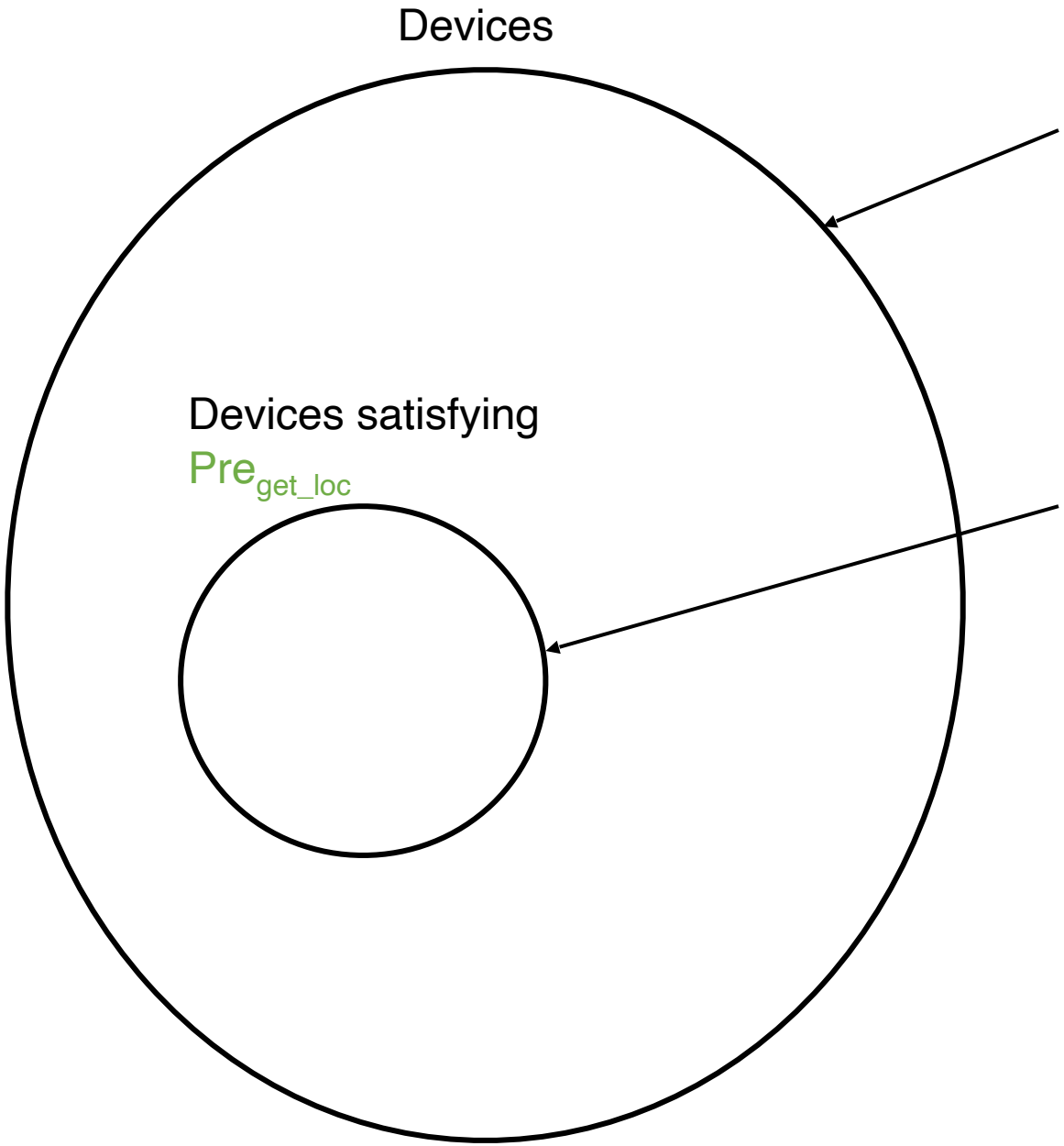
“you can only get_loc a device you own”

$Pre_{get_loc}(d)$ =

1. d in Device
2. you own d

Guarantee = all d in Device: $Post_{get_loc}(d, loc)$

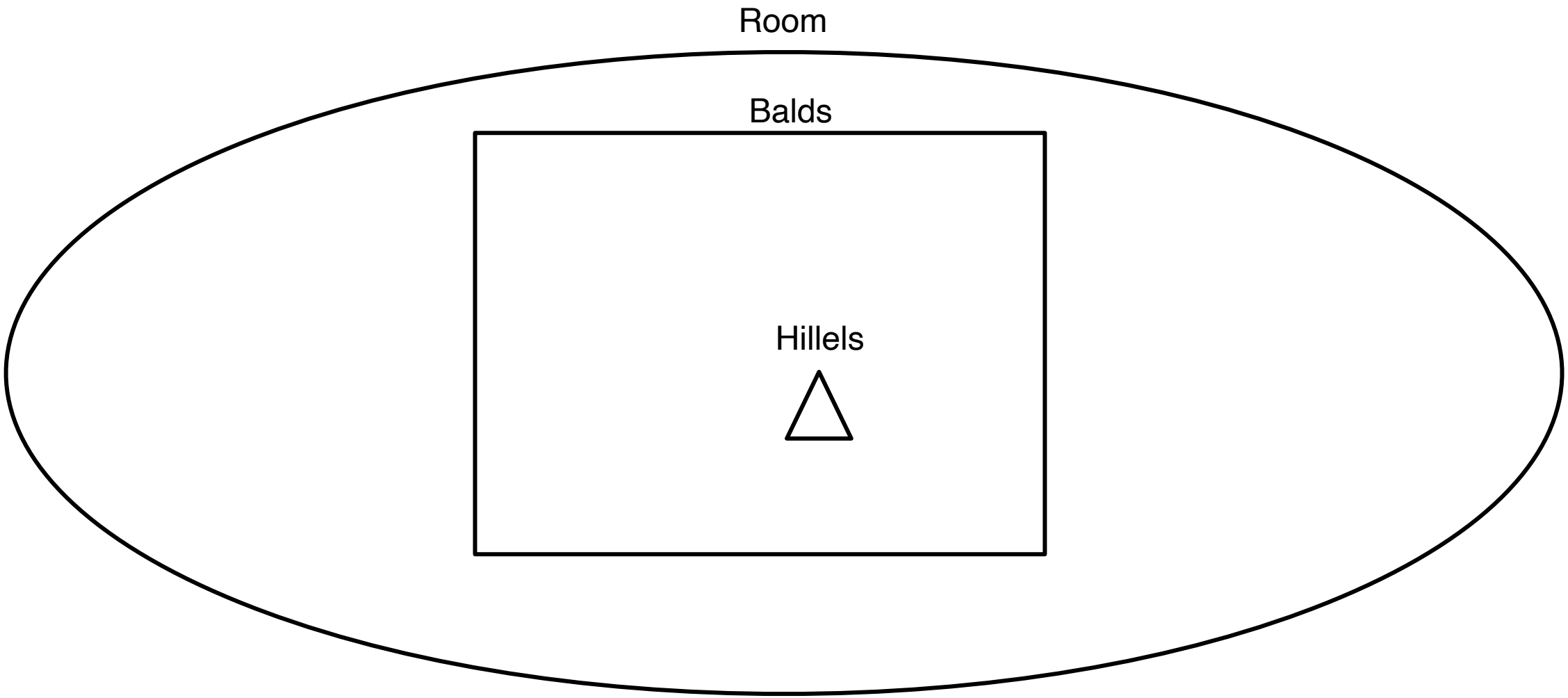
$Pre_{get_loc}(d)$??



all d in Device: $Post_{get_loc}(d, loc)$



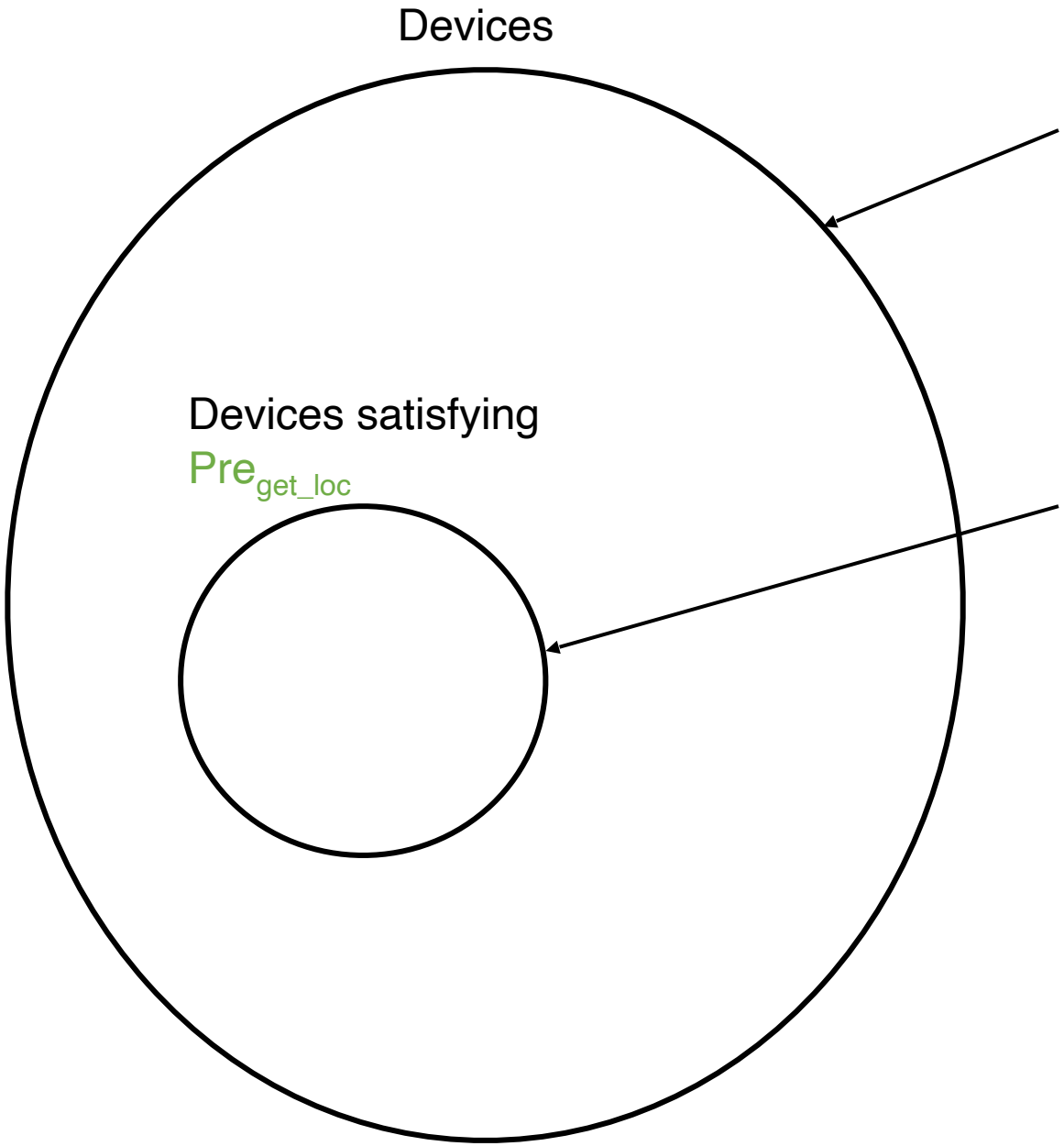
all d in {d in Device: $Pre_{get_loc}(d)$}:
$Post_{get_loc}(d, loc)$



{p in Room: Hillel(p)} subset {p in Room: Bald(p)}

all p in {p in Room: Hillel(p)}: Bald(p)

all p in Room: Hillel(p) => Bald(p)



all d in Device: $Post_{get_loc}(d, loc)$



all d in {d in Device: $Pre_{get_loc}(d)$}:
$Post_{get_loc}(d, loc)$



all d in Device:
$Pre_{get_loc}(d) \Rightarrow Post_{get_loc}(d, loc)$

Preconditions + Postconditions = Specs

f :: S -> T

$Spec_f$ = all x in S: $Pre_f(x)$ => $Post_f(x, f(x))$

$Spec_f = Pre_f \Rightarrow Post_f$

Example of Pre and Post Conditions

div(a, b) = a/b

all a, b in Number:

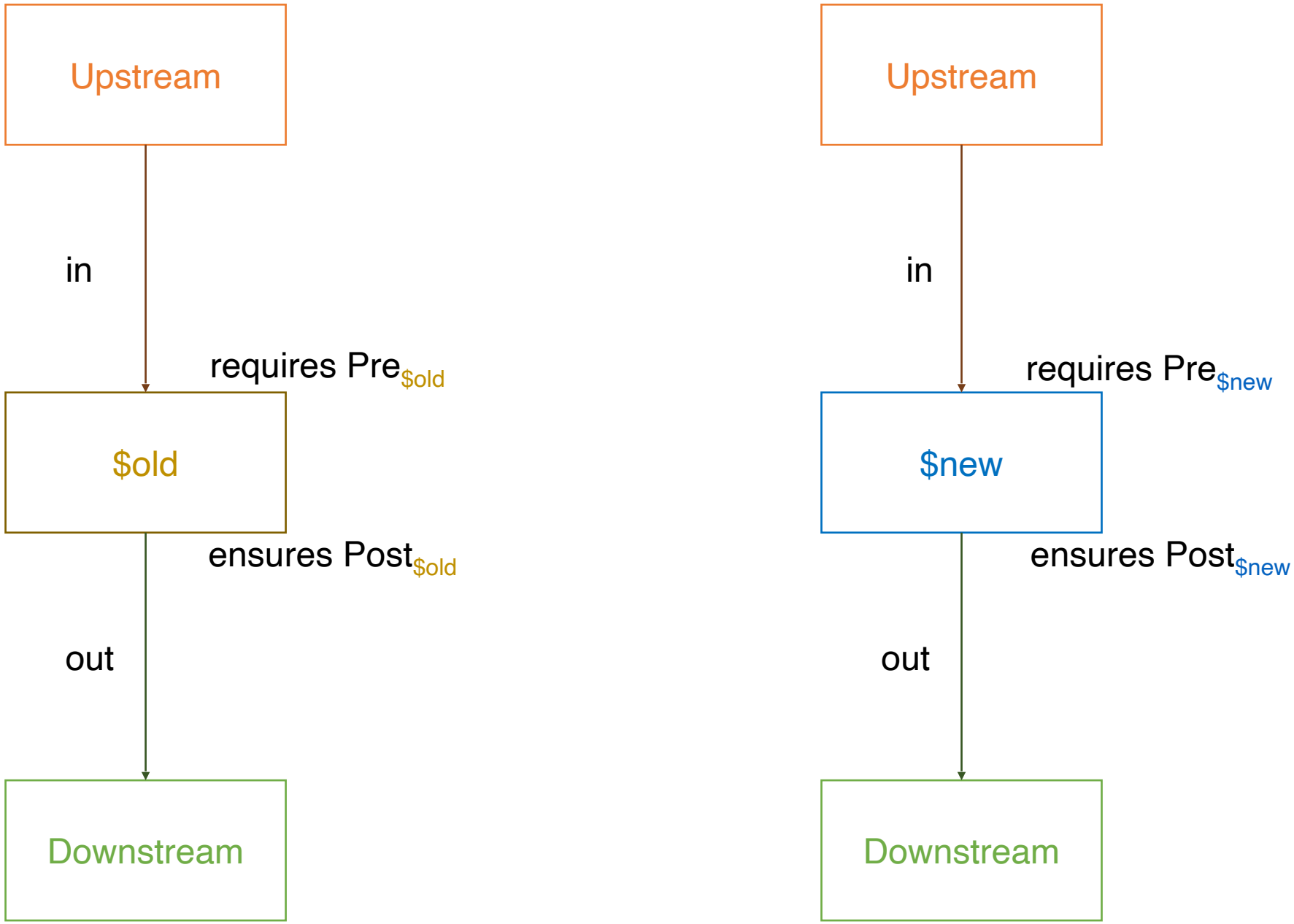
b ≠ 0 => (a/b)*b == a

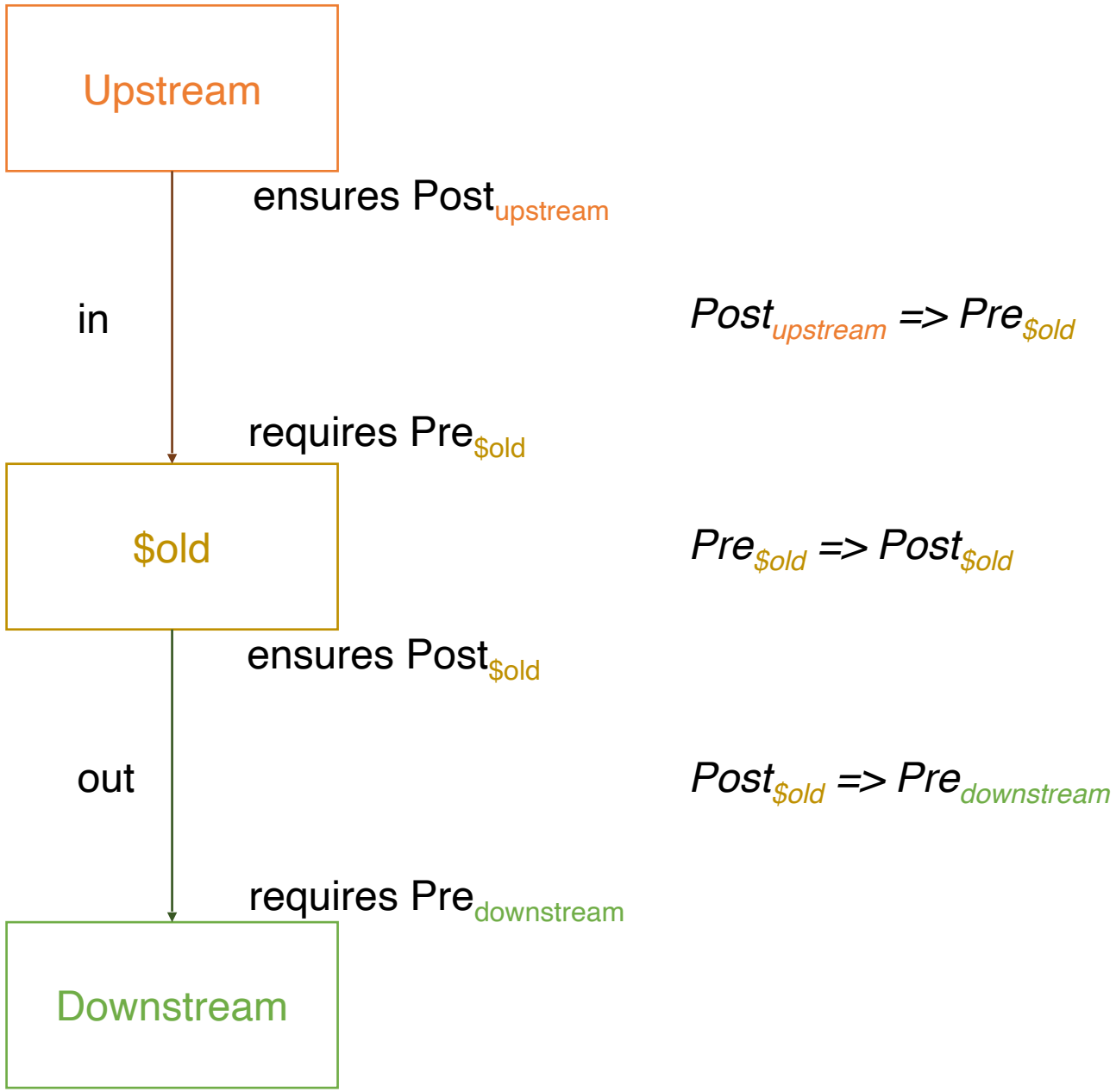
$Pre_{div}(a, b) = b \neq 0$

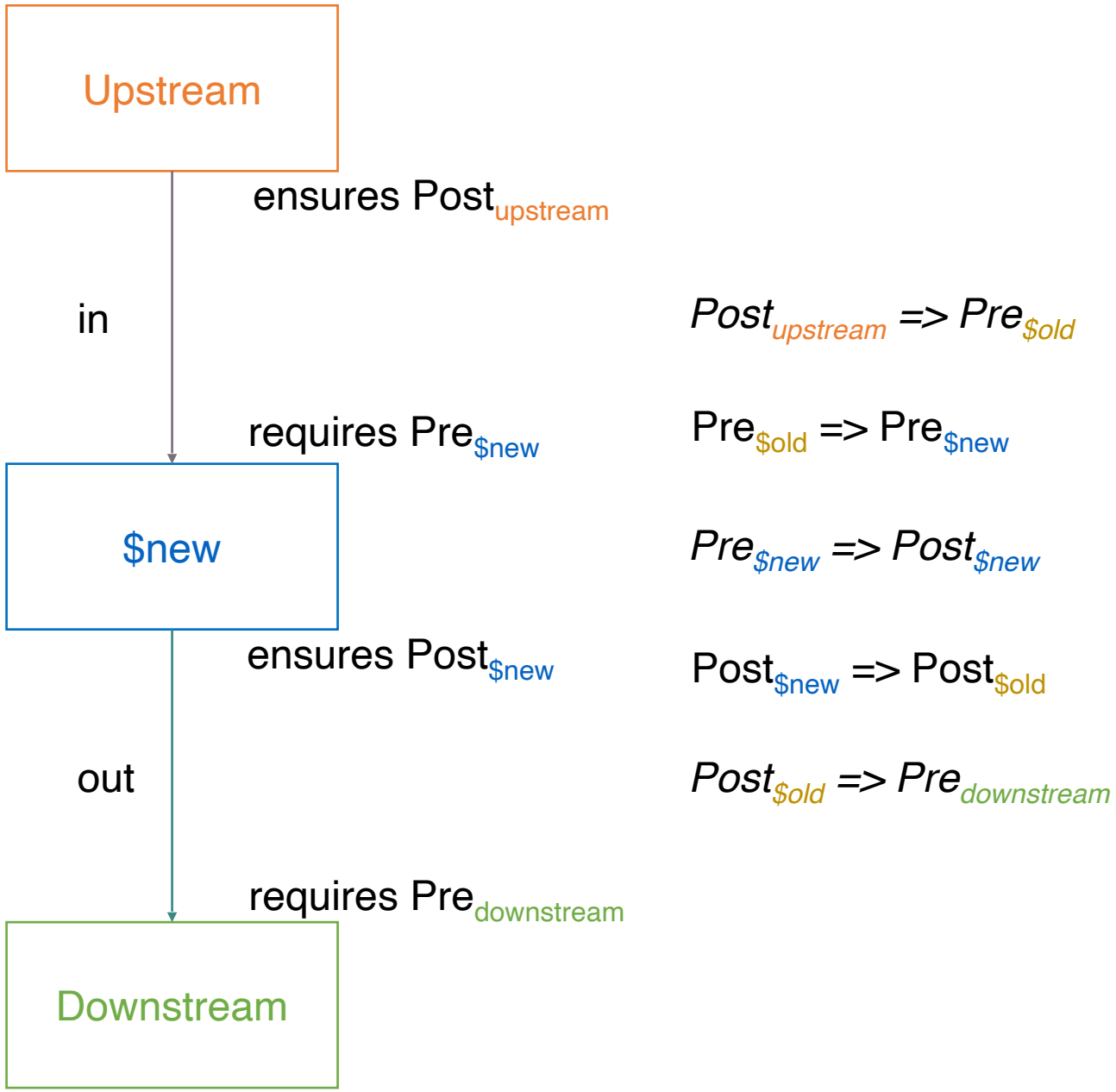
$Post_{div}(a, b, out) = out * b == a$

$$Spec_{\$old} = Pre_{\$old} \Rightarrow Post_{\$old}$$

$$Spec_{\$new} = Pre_{\$new} \Rightarrow Post_{\$new}$$


Upstream
in
requires Pre$old
$old
ensures Post$old
out
Downstream
Upstream
in
requires Pre$new
$new
ensures Post$new
out
Downstream


Upstream
ensures Post upstream
in
Post upstream => Pre $old
requires Pre $old
$old
Pre $old => Post $old
ensures Post $old
out
Post $old => Pre downstream
requires Pre downstream
Downstream

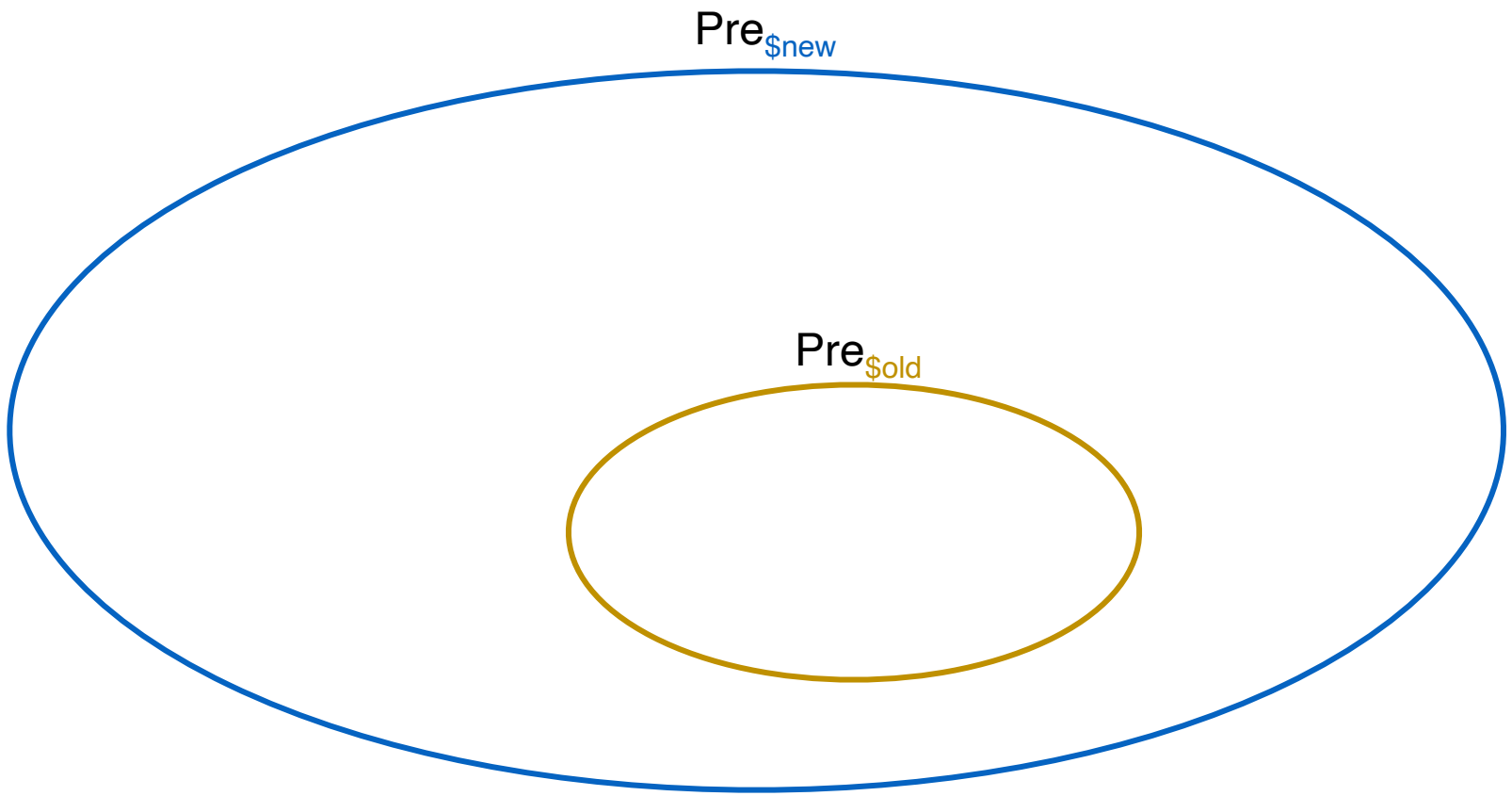

Upstream
ensures Post upstream
in
Post upstream => Pre $old
requires Pre $new
Pre $old => Pre $new
$new
Pre $new => Post $new
ensures Post $new
Post $new => Post $old
out
Post $old => Pre downstream
requires Pre downstream
Downstream

$new can replace $old when

1. $Pre_{\$old} \Rightarrow Pre_{\$new}$

2. $Post_{\$new} \Rightarrow Post_{\$old}$

$Pre_{\$old} \Rightarrow Pre_{\$new}$



$new can replace $old when

1. $Pre_{\$old} \Rightarrow Pre_{\$new}$
2. $Post_{\$new} \Rightarrow Post_{\$old}$

1. You keep or loosen preconditions
2. You keep or tighten postconditions

get_loc :: Device -> Location

$Pre_{\$old}(d)$

you own device d

$Pre_{mium}(d)$

you own device d
AND have a premium account

$Pre_{\$old} \Rightarrow Pre_{mium}$?

get_loc :: Device -> Location

$Pre_{\$old}(d)$

you own device d

$Pre_{cent}(d)$

you own device d

AND d was made last year

$$Pre_{\$old} \Rightarrow Pre_{cent} ?$$

get_loc :: Device -> Location

Pre$old(d)

you own device d

Prelated(d)

you or someone in your family owns device d

Pre$old => Prelated?

get_loc :: Device -> Location

$Pre_{\$old}(d)$

you own device d

$Pre_{vil}(d)$

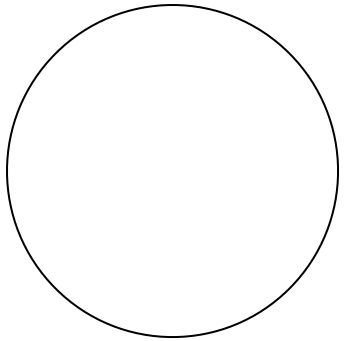
you own device d OR have a premium account

$$Pre_{\$old} \Rightarrow Pre_{vil}?$$

get_loc :: Device -> Location

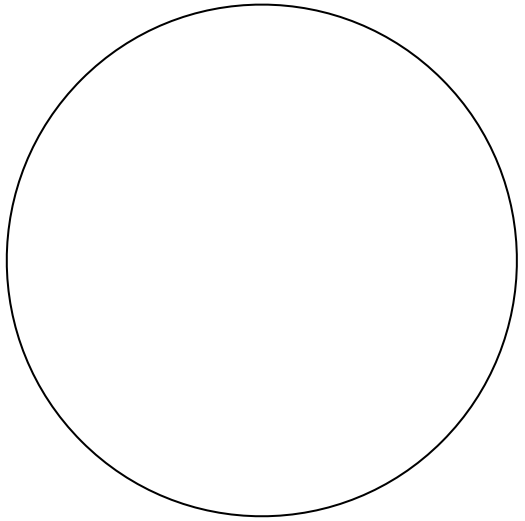
$Post_{\$old}$(d, loc)

1. LocatedIn(d, loc)
2. area(loc) ≤ deciacre



$Post_{wide}$(d, loc)

1. LocatedIn(d, loc)
2. area(loc) ≤ acre



$Post_{wide}$ => $Post_{\$old}$?

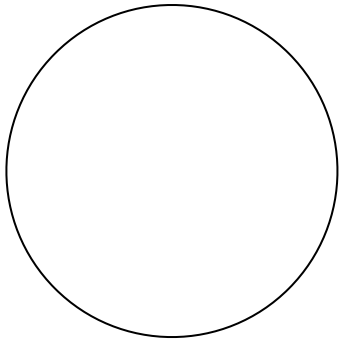
get_loc :: Device -> Location

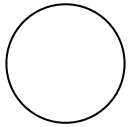
$Post_{\$old}$(d, loc)

1. LocatedIn(d, loc)
2. area(loc) ≤ deciacre

$Post_{justwrong}$(d, loc)

1. area(loc) ≤ centiacre





$Post_{wide}$ => $Post_{\$old}$?

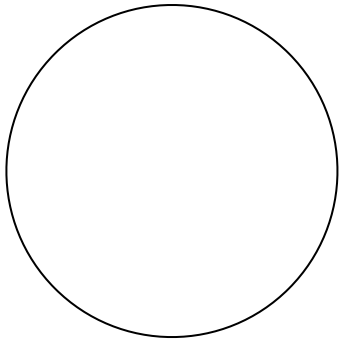
get_loc :: Device -> Location

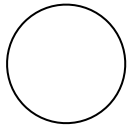
$Post_{\$old}$(d, loc)

1. LocatedIn(d, loc)
2. area(loc) ≤ deciacre

$Post_{accurate}$(d, loc)

1. LocatedIn(d, loc)
2. area(loc) ≤ centiacre

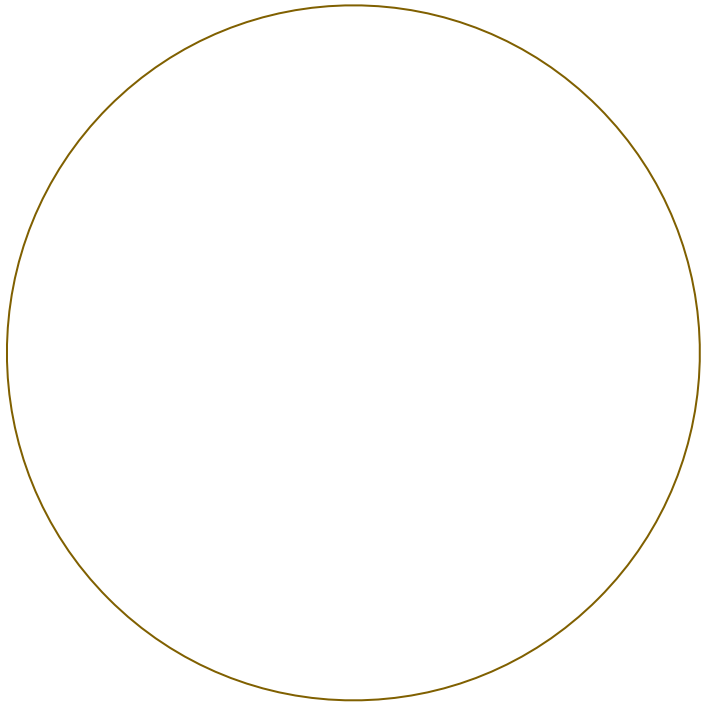




$Post_{accurate}$ => $Post_{\$old}$?

$old

1 deciacre



$new

1 centiacre



get_loc :: Device -> Location

$Post_{\$old}$(d, loc)

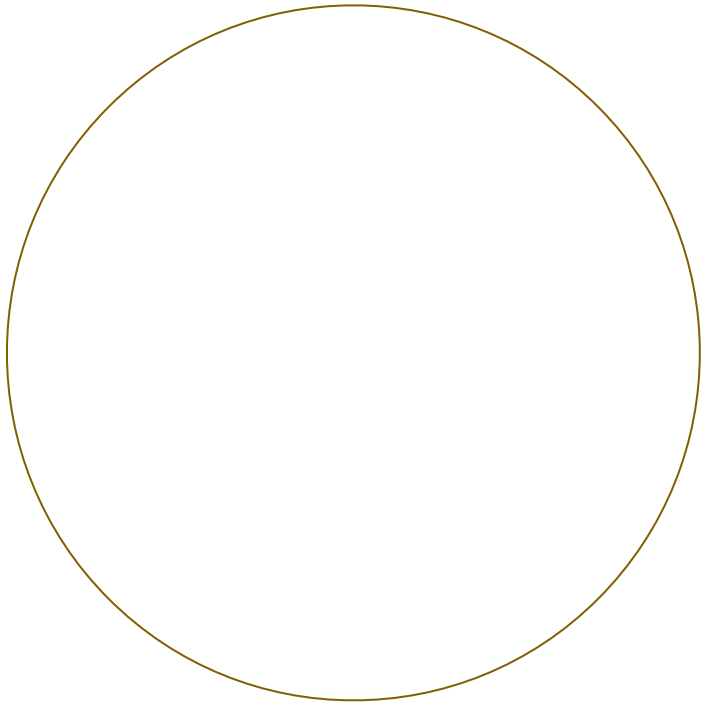
1. LocatedIn(d, loc)
2. area(loc) ≤ deciacre

$Post_{\$new}$(d, loc) =

1. LocatedIn(d, loc)
2. area(loc) ≤ centiacre

$Post_{\$new}$ => $Post_{\$old}$?

$old



$new



Not all properties are postconditions.

$Post_{get_loc}$

1. LocatedIn(d, loc)
2. area(loc) ≤ 1 deciacre

$Properties_{get_loc}$

1. $Post_{get_loc}$
2. area is a circle
3. area is convex
4. area is not disjoint
5. response comes in ≤ 20 ms
6. area EXACTLY 1 deciacre???

With a sufficient number of users of an API, it does not matter what you promise in the contract: all observable behaviors of your system will be depended on by somebody.

Hyrum Wright

What about state?





Barbara Liskov

QED

If for each object o_1 of type S there is an object o_2 of type T such that for all programs P defined in terms of T, the behavior of P is unchanged when o_1 is substituted for o_2, then S is a subtype of T.

—Barbara Liskov, “Data abstraction and hierarchy”

“A Behavioral Notion of Subtyping”

DEFINITION OF THE SUBTYPE RELATION, $<:$ $\sigma = \langle O_\sigma, S, M\rangle$ is a *subtype* of $\tau = \langle O_\tau, T, N\rangle$ if there exists an abstraction function, $A : S \to T$, and a renaming map, $R : M \to N$, such that:

(1) The abstraction function respects invariants:
—*Invariant Rule.* $\forall s : S \;.\; I_\sigma(s) \Rightarrow I_\tau(A(s))$
A may be partial, need not be onto, but can be many-to-one.

(2) Subtype methods preserve the supertype methods' behavior. If m_τ of τ is the corresponding renamed method m_σ of σ, the following rules must hold:
—*Signature rule.*
—*Contravariance of arguments.* m_τ and m_σ have the same number of arguments. If the list of argument types of m_τ is α_i and that of m_σ is β_i, then $\forall i \;.\; \alpha_i < \beta_i$.
—*Covariance of result.* Either both m_τ and m_σ have a result or neither has. If there is a result, let m_τ's result type be α and m_σ's be β. Then $\beta < \alpha$.
—*Exception rule.* The exceptions signaled by m_σ are contained in the set of exceptions signaled by m_τ.
—*Methods rule.* For all $x : \sigma$:
—*Pre-condition rule.* $m_\tau.pre[A(x_{pre})/x_{pre}] \Rightarrow m_\sigma.pre$.
—*Post-condition rule.* $m_\sigma.post \Rightarrow m_\tau.post[A(x_{pre})/x_{pre}, A(x_{post})/x_{post}]$

(3) Subtype constraints ensure supertype constraints.
—*Constraint Rule.* For all computations, c, and all states ρ and ψ in c such that ρ precedes ψ, for all $x : \sigma$:
$C_\sigma \Rightarrow C_\tau[A(x_\rho)/x_\rho, A(x_\psi)/x_\psi]$

Fig. 4. Definition of the Subtype Relation (Constraint Rule)

1. $Pre_{\$old} \Rightarrow Pre_{\$new}$
2. $Post_{\$new} \Rightarrow Post_{\$old}$

Things Barbara Liskov Invented



- Consensus protocols
- Abstract Data Types
- Iterators
- Generators
- Object-Oriented Programming
- Behavioral Subtyping
- Replacement

Summary

- Logic is useful
- With a simple bit of math, we can develop quite complex topics
- We can safely replace functions by loosening preconditions and tightening postconditions
- Barbara Liskov extends that to types

www.logicforprogrammers.com

